JAVA PROGRAMMING

Assignment Answers

Object-Oriented Programming • Exception Handling • Collections Framework • Multithreading • Polymorphism

Five long-answer questions with explanations, examples, and Java code

Question 1. Explain the four fundamental principles of Object-Oriented Programming (OOP): encapsulation, inheritance, polymorphism, and abstraction. Describe each principle with suitable real-world examples and Java code.

Answer: Object-Oriented Programming (OOP) is a programming approach based on classes and objects. It helps developers write programs that are modular, reusable, secure, and easier to maintain. Java supports four fundamental OOP principles: encapsulation, inheritance, polymorphism, and abstraction.

1. Encapsulation

Encapsulation combines data and methods into a single unit called a class. It also restricts direct access to data using access modifiers such as `private` and provides controlled access through public methods.

Real-world example: A bank account protects its balance from being changed directly. Users deposit or withdraw money through authorized operations.

```
class BankAccount {
    private double balance;

    public void deposit(double amount) {
        if (amount &gt; 0) {
            balance += amount;
        }
    }

    public double getBalance() {
        return balance;
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();
        account.deposit(5000);
        System.out.println(account.getBalance());
    }
}
```

Output: 5000.0

Here, `balance` is private and cannot be accessed directly outside the class. The methods provide controlled access.

Advantages: Protects data, improves security and data integrity, and makes code easier to maintain.

2. Inheritance

Inheritance allows one class to acquire the properties and methods of another class. The existing class is the parent or superclass, and the new class is the child or subclass. Java uses the `extends` keyword for class inheritance.

Real-world example: Cars and motorcycles are vehicles. They share common features, such as starting, while also having their own specific features.

```
class Vehicle {
    void start() {
        System.out.println("Vehicle is starting");
    }
}

class Car extends Vehicle {
    void drive() {
        System.out.println("Car is moving");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Car car = new Car();
        car.start();
        car.drive();
    }
}
```

Output: Vehicle is starting
Car is moving

The `Car` class inherits the `start()` method and defines its own `drive()` method.

Advantages: Promotes code reuse, reduces duplication, and establishes relationships between classes.

3. Polymorphism

Polymorphism means “many forms.” It allows the same method name or interface to represent different behaviors. Java supports compile-time polymorphism through method overloading and runtime polymorphism through method overriding.

Real-world example: A mobile phone can make calls, take photographs, or play music depending on the operation selected.

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Cat meows");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a;

        a = new Dog();
        a.sound();

        a = new Cat();
        a.sound();
    }
}
```

Output: Dog barks
Cat meows

Although the reference variable has type `Animal`, the executed method depends on the actual object.

Advantages: Improves flexibility, supports extensible design, and reduces repetitive conditional logic.

4. Abstraction

Abstraction hides implementation details and exposes essential functionality. In Java, it is achieved mainly through abstract classes and interfaces.

Real-world example: An ATM lets a customer withdraw cash or check a balance without exposing the internal banking operations.

```
abstract class Shape {
    abstract double calculateArea();
}

class Circle extends Shape {
    double radius = 5;

    @Override
    double calculateArea() {
        return Math.PI * radius * radius;
    }
}

public class Main {
    public static void main(String[] args) {
        Shape shape = new Circle();
        System.out.println(shape.calculateArea());
    }
}
```

Output: 78.53981633974483

The abstract class declares the required operation, while `Circle` implements it.

Advantages: Hides unnecessary details, reduces complexity, and supports maintainable code.

Conclusion: Encapsulation protects data, inheritance promotes reusability, polymorphism provides multiple behaviors, and abstraction hides implementation details. Together, these principles form the foundation of OOP in Java.

Question 2. What is exception handling in Java? Explain try, catch, finally, throw, and throws with suitable examples. Also differentiate between checked and unchecked exceptions.

Answer: Exception handling is a Java mechanism for detecting and managing runtime problems so that a program can respond appropriately instead of terminating unexpectedly. An exception is an event that interrupts the normal flow of execution.

1. The try and catch blocks

The `try` block contains code that may generate an exception. It must be followed by at least one `catch` block or a `finally` block. A `catch` block handles a matching exception.

```
public class Main {
    public static void main(String[] args) {
        try {
            int result = 10 / 0;
            System.out.println(result);
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero");
        }
    }
}
```

Output: Cannot divide by zero

Division by zero for integer values throws an `ArithmeticException`, which is handled by the catch block.

2. The finally block

The `finally` block normally executes whether an exception occurs or not. It is commonly used for cleanup. It may not execute if the JVM terminates abruptly.

```
public class Main {
    public static void main(String[] args) {
        try {
            int result = 10 / 2;
            System.out.println(result);
        } catch (ArithmeticException e) {
            System.out.println("Exception occurred");
        } finally {
            System.out.println("Execution completed");
        }
    }
}
```

Output: 5
Execution completed

3. The throw keyword

The `throw` keyword explicitly throws an exception object. It can be used when the program detects an invalid condition.

```
public class Main {
    public static void main(String[] args) {
        int age = 16;

        if (age &amp;lt; 18) {
            throw new IllegalArgumentException(
                "Age must be at least 18"
            );
        }

        System.out.println("Eligible");
    }
}
```

This throws an `IllegalArgumentException`. Because it is not caught in this example, the program ends with an exception message.

4. The throws keyword

The `throws` keyword declares exceptions that a method may pass to its caller. It is particularly important for checked exceptions.

```
import java.io.FileReader;
import java.io.IOException;

public class Main {
    static void readFile() throws IOException {
        FileReader file = new FileReader("data.txt");
        file.close();
    }

    public static void main(String[] args) {
        try {
            readFile();
            System.out.println("File operation completed");
        } catch (IOException e) {
            System.out.println("File error: " + e.getMessage());
        }
    }
}
```

Opening or closing the file may cause an `IOException`. The method declares the exception, and the caller handles it.

Difference between throw and throws

throw	throws
Explicitly throws an exception object.	Declares possible exceptions in a method signature.
Used inside a method or block.	Used in a method declaration.
Followed by an exception object.	Followed by exception class names.
Throws one exception object at a time.	Can declare multiple exception types.

Checked vs. unchecked exceptions

Checked exceptions	Unchecked exceptions
Checked by the compiler at compile time.	Not required to be caught or declared by the compiler.
Must be caught or declared with throws.	Handling or declaring them is optional.
Often represent conditions an application may recover from.	Often indicate programming errors or invalid operations.
Examples: IOException, SQLException, ClassNotFoundException.	Examples: NullPointerException, ArithmeticException, ArrayIndexOutOfBoundsException.

Checked exceptions generally extend `Exception` but not `RuntimeException`. Unchecked exceptions include `RuntimeException` and its subclasses. `Error` types are also unchecked in Java, but they are not ordinary exceptions.

Conclusion: The try and catch blocks manage exceptions, finally supports cleanup, throw explicitly raises an exception, and throws declares exceptions that may be passed to the caller.

Question 3. Explain the Java Collections Framework in detail. Discuss the List, Set, Queue, and Map interfaces, along with their implementations, characteristics, and practical applications.

Answer: The Java Collections Framework (JCF) is a set of interfaces, classes, and algorithms used to store, retrieve, and manipulate groups of objects. It is available mainly through the `java.util` package and provides reusable data structures for common operations.

1. List interface

A `List` stores elements in a sequence, preserves positional order, and allows duplicates. Elements can be accessed by index.

• **ArrayList:** Resizable array; fast indexed access and suitable for frequent reading.

• **LinkedList:** Linked nodes; supports efficient insertion or removal at known positions, but indexed access is slower.

• **Vector:** Legacy resizable array whose individual methods are synchronized.

```
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List&lt;String&gt; names = new ArrayList&lt;&gt;();
        names.add("Aman");
        names.add("Riya");
        names.add("Aman");

        System.out.println(names);
        System.out.println(names.get(1));
    }
}
```

Output: [Aman, Riya, Aman]
Riya

The duplicate name is allowed, and `get(1)` retrieves the element at index 1. Applications include student records, shopping-cart items, playlists, and ordered search results.

2. Set interface

A `Set` stores unique elements and does not allow duplicates according to its equality rules. Ordering depends on the implementation.

• **HashSet:** Uses hashing and does not guarantee iteration order.

• **LinkedHashSet:** Maintains insertion order during iteration.

• **TreeSet:** Stores elements in sorted order using natural ordering or a comparator.

```
import java.util.HashSet;
import java.util.Set;

public class Main {
    public static void main(String[] args) {
        Set&lt;Integer&gt; numbers = new HashSet&lt;&gt;();
        numbers.add(10);
        numbers.add(20);
        numbers.add(10);

        System.out.println(numbers);
    }
}
```

Possible output: [20, 10]

The exact order may vary because HashSet does not guarantee iteration order. The duplicate value 10 is stored only once. Applications include removing duplicate records, maintaining unique user IDs, and checking membership.

3. Queue interface

A `Queue` holds elements for processing. Many queues follow FIFO (First In, First Out), while priority-based queues use a different ordering rule.

• **LinkedList:** Can be used as a queue or deque.

• **PriorityQueue:** Retrieves elements according to priority; by default, the smallest element comes first under natural ordering.

• **ArrayDeque:** Resizable-array implementation suitable for queue and stack operations.

```
import java.util.LinkedList;
import java.util.Queue;

public class Main {
    public static void main(String[] args) {
        Queue&lt;String&gt; queue = new LinkedList&lt;&gt;();
        queue.offer("A");
        queue.offer("B");
        queue.offer("C");

        System.out.println(queue.poll());
        System.out.println(queue);
    }
}
```

Output: A
[B, C]

The `offer()` method inserts an element, while `poll()` retrieves and removes the queue head. Applications include task scheduling, printer queues, customer service systems, and breadth-first search.

4. Map interface

A `Map` stores key-value pairs. Keys are unique, but different keys may map to equal values. Map is part of the Collections Framework, but it does not extend the `Collection` interface.

• **HashMap:** Hash-based storage without guaranteed iteration order.

• **LinkedHashMap:** Maintains a defined iteration order, usually insertion order by default.

• **TreeMap:** Stores keys in sorted order.

• **Hashtable:** Legacy synchronized implementation that does not allow null keys or values.

```
import java.util.HashMap;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        Map&lt;Integer, String&gt; students = new HashMap&lt;&gt;();
        students.put(101, "Aman");
        students.put(102, "Riya");
        students.put(103, "Karan");

        System.out.println(students.get(102));
        System.out.println(students.containsKey(101));
    }
}
```

Output: Riya
true

The `put()` method inserts or updates a mapping, `get()` retrieves a value, and `containsKey()` checks whether a key exists. Applications include employee records, product inventories, phone directories, and

caches.

Comparison of collection interfaces

Interface	Duplicates	Ordering	Common implementations
List	Allowed	Positional order	ArrayList, LinkedList
Set	Not allowed	Depends on implementation	HashSet, LinkedHashSet, TreeSet
Queue	Usually allowed	FIFO or priority-based	LinkedList, PriorityQueue, ArrayDeque
Map	Keys unique; values may repeat	Depends on implementation	HashMap, LinkedHashMap, TreeMap

Conclusion: List is useful for ordered elements, Set for uniqueness, Queue for processing tasks, and Map for key-value associations. Choosing an appropriate implementation improves performance and maintainability.

Question 4. What is multithreading in Java? Explain how threads are created using the Thread class and Runnable interface. Discuss thread lifecycle, synchronization, race conditions, and the advantages of multithreading.

Answer: Multithreading is the execution of multiple threads within a single program. A thread is a lightweight unit of execution. Threads can make progress concurrently and may execute in parallel when multiple processor cores are available. Multithreading is used in web servers, games, background processing, and interactive applications.

1. Creating a thread using the Thread class

One way to create a thread is to extend the `Thread` class and override its `run()` method. Calling `start()` begins a new thread of execution, which then invokes `run()`.

```
class MyThread extends Thread {
    @Override
    public void run() {
        for (int i = 1; i &lt;= 5; i++) {
            System.out.println("Thread: " + i);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();
    }
}
```

The thread prints numbers from 1 to 5. Exact timing and output order are not guaranteed because scheduling is controlled by the JVM and operating system.

2. Creating a thread using Runnable

Another approach is to implement `Runnable` and pass its object to a `Thread`. This separates the task from the thread and allows the class to extend another class.

```
class MyTask implements Runnable {
    @Override
    public void run() {
        System.out.println("Task is running");
    }
}

public class Main {
    public static void main(String[] args) {
        Thread t = new Thread(new MyTask());
        t.start();
    }
}
```

Output: Task is running

Thread class	Runnable interface
The class extends Thread.	The class implements Runnable.
Task and thread are combined in the same class.	Task is separated from the thread.
The class cannot extend another class.	The class can extend another class.
Useful for simple examples or specialized thread behavior.	Often preferred for reusable tasks and executor services.

3. Thread lifecycle

Java defines six thread states in the `Thread.State` enum:

• **NEW:** The thread object is created, but start() has not been called.

• **RUNNABLE:** The thread is ready to run or is running. Java does not define a separate RUNNING state.

• **BLOCKED:** The thread is waiting to acquire a monitor lock.

• **WAITING:** The thread is waiting indefinitely for another action.

• **TIMED_WAITING:** The thread is waiting for a specified period.

• **TERMINATED:** The run() method has completed or ended because of an uncaught exception.

A thread normally moves from NEW to RUNNABLE after start(). It may enter BLOCKED, WAITING, or TIMED_WAITING depending on what it is waiting for, and eventually becomes TERMINATED.

4. Synchronization

Synchronization controls access to shared resources by multiple threads. The `synchronized` keyword allows only one thread at a time to execute a synchronized method or block protected by the same monitor lock.

```
class Counter {
    private int count = 0;

    synchronized void increment() {
        count++;
    }

    int getCount() {
        return count;
    }
}

public class Main {
    public static void main(String[] args)
            throws InterruptedException {
        Counter c = new Counter();

        Thread t1 = new Thread(() -&gt; {
            for (int i = 0; i &lt; 1000; i++) c.increment();
        });

        Thread t2 = new Thread(() -&gt; {
            for (int i = 0; i &lt; 1000; i++) c.increment();
        });

        t1.start();
        t2.start();
        t1.join();
        t2.join();

        System.out.println(c.getCount());
    }
}
```

Output: 2000

Synchronization protects the shared counter from simultaneous updates. The `join()` calls make the main thread wait for both threads to finish before printing the result.

5. Race conditions

A race condition occurs when multiple threads access shared data and the result depends on the timing or order of their operations. The statement `count++` involves reading, incrementing, and writing the value. Without synchronization or another suitable concurrency mechanism, updates can be lost.

```
class Counter {
    int count = 0;
```

```
    void increment() {
        count++;
    }
}
```

If multiple threads call `increment()` on the same object, the final count may be lower than expected. Synchronization, `AtomicInteger`, or a suitable lock can prevent this problem.

6. Advantages of multithreading

• **Responsiveness:** The application can handle user input while background tasks run.

• **CPU utilization:** Independent tasks can run in parallel on multicore processors.

• **Concurrent processing:** Multiple requests can be processed within one application.

• **Resource sharing:** Threads within a process can share memory and other resources.

• **Throughput:** Well-designed applications may complete more work in a given time.

Multithreading can also introduce synchronization overhead, race conditions, deadlocks, and debugging difficulties.

Conclusion: Multithreading enables concurrent execution in Java. Understanding thread creation, lifecycle, synchronization, and race conditions is essential for writing correct concurrent applications.

Question 5. Explain method overloading and method overriding in Java. Differentiate between compile-time and runtime polymorphism with appropriate code examples.

Answer: Method overloading and method overriding are important Java features that support polymorphism. They allow methods to behave differently according to their parameter lists or the object involved.

1. Method overloading

Method overloading occurs when a class contains multiple methods with the same name but different parameter lists. Methods may differ in the number of parameters, parameter types, or parameter order. The compiler selects the applicable overloaded method, so overloading is an example of compile-time polymorphism.

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }

    double add(double a, double b) {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator c = new Calculator();

        System.out.println(c.add(10, 20));
        System.out.println(c.add(10, 20, 30));
        System.out.println(c.add(2.5, 3.5));
    }
}
```

Output: 30
60
6.0

The three methods are named `add`, but their parameter lists differ. Changing only the return type does not create a valid overload; the parameter list must differ.

2. Method overriding

Method overriding occurs when a subclass provides its own implementation of an inherited instance method with a compatible signature. For an overridden instance method, the implementation is selected at runtime according to the actual object's class.

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Cat meows");
    }
```

```
}

public class Main {
    public static void main(String[] args) {
        Animal a1 = new Dog();
        Animal a2 = new Cat();

        a1.sound();
        a2.sound();
    }
}
```

Output: Dog barks
Cat meows

Both variables are declared as `Animal`, but the actual objects determine which overridden implementation executes.

3. Compile-time vs. runtime polymorphism

Compile-time polymorphism: Commonly achieved through overloading. The compiler selects an applicable method using the argument types and overload-resolution rules.

Runtime polymorphism: Achieved through overriding. The implementation of an overridden instance method is selected according to the actual object type at runtime.

Method overloading	Method overriding
Usually within the same class; inherited methods may also be overloaded.	Requires inheritance or another valid subtype relationship.
Same name with different parameter lists.	Same or compatible method signature in the subclass.
Compile-time polymorphism.	Runtime polymorphism.
Selected using compile-time argument types and overload rules.	Selected using the actual object's class.
Return types may differ if parameter lists differ and declarations are valid.	Return type must be the same or covariant for reference types.
Static methods can be overloaded.	Static methods are hidden, not overridden.

4. Combined example

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }

    void eat(String food) {
        System.out.println("Eating " + food);
    }

    void eat(String food, int quantity) {
        System.out.println(
            "Eating " + quantity + " units of " + food
        );
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a = new Dog();

        // Runtime polymorphism: overriding
        a.sound();

        // Compile-time polymorphism: overloading
        a.eat("bread");
        a.eat("bread", 2);
    }
}
```

Output: Dog barks
Eating bread
Eating 2 units of bread

The `sound()` call demonstrates runtime polymorphism because Dog overrides the method inherited from Animal. The two `eat()` methods demonstrate compile-time polymorphism because they have different parameter lists.

Conclusion: Overloading lets the same method name perform different operations depending on the arguments. Overriding lets subclasses customize inherited behavior. Both improve flexibility, reusability, and extensibility in Java.